Conversation AI Copilot

Complete Technical Architecture, Implementation & Systems Engineering Report

ORGANIZATION	LEAD DEVELOPER	PLATFORM INTEGRATION	CORE RUNTIME
XortLogix	Muhammad Okasha	GoHighLevel API 2.0	Python 3.10+ / FastAPI

1. Project Overview & Business Purpose

Conversation AI Copilot is an enterprise-grade artificial intelligence co-pilot and autonomous CRM execution system built by **Muhammad Okasha** under **XortLogix**. The platform is engineered specifically for digital agencies, SaaS operators, and high-ticket marketing teams working on the **GoHighLevel (HighLevel / GHL)** CRM ecosystem.

Traditional chatbots operate as passive text generators with no awareness of live CRM data. Conversation AI Copilot bridges this gap by functioning simultaneously as:

- **An Autonomous GHL Operations Engineer:** Capable of authenticating into HighLevel Sub-Accounts and executing live write/read operations (Contacts, Pipelines, Tags, Opportunities, Custom Fields) via native LLM Function Calling.
- **A Full-Stack Conversion Funnel Architect:** Capable of writing complete, self-contained single-file HTML/Tailwind CSS interactive applications featuring multi-step VSL rooms, true 2-step checkout forms with Luhn credit card validation, and drop-off recovery sequences.
- **A Multi-Provider Resilient Gateway:** Orchestrating 11 API keys across Google Gemini, Groq Cloud, and OpenRouter with real-time sliding-window TPM tracking and seamless mid-stream handovers.

2. Complete Codebase Directory & File Manifest

The codebase is organized into modular Python backend services and a vanilla JavaScript frontend designed without framework bloat for maximum execution speed:

File / Directory	Primary Module	Detailed Purpose & Implementation Specifics
`app.py`	FastAPI Web Server	Application entry point, HTTP routing, SSE streaming generator (`/api/chat-agent`), thread persistence, CORS configuration, static asset mounting, and singleton engine caching (`_get_engine()`).
`agent_engine.py`	AI Execution Engine	Core orchestration logic (1,750+ lines). Houses prompt intent classification, Google Gemini streaming, OpenAI/Groq streaming, GHL tool calling dispatcher, truncation detection, and mid-stream cross-model handovers.
`key_pool_manager.py`	Key Pool Rotation	Thread-safe multi-key rotation classes: `GeminiKeyPool` (managing 5 Google Gemini keys) and `OpenRouterKeyPool` (managing 6 OpenRouter keys). Implements automatic 429 exponential backoff, circuit-breaking, and recovery timers.
`ghl_client.py`	GoHighLevel Client	Comprehensive wrapper around HighLevel API 2.0 (version `2021-07-28`). Manages Bearer token authorization, Location ID validation, and CRUD operations for contacts, pipelines, tags, and custom fields.
`usage_tracker.py`	Real-Time Metrics	Implements a 60-second sliding-window token usage algorithm that accurately calculates live Tokens Per Minute (TPM), Requests Per Minute (RPM), daily token consumption, and percentage capacity per model.
`static/index.html`	Web Interface Structure	Single-page responsive application featuring the floating input island, 6 suggestion chips, the 6-step Smart Asset Wizard modal, confirmation popups, and live quota badges.
`static/style.css`	Design System & CSS	Over 3,800 lines of custom vanilla CSS. Implements a dark/light mode design system, glassmorphism backdrop filters, squircle send buttons, responsive breakpoints, and syntax highlighting.
`static/app.js`	Frontend Controller	Client-side runtime (4,400+ lines). Handles Server-Sent Events (SSE) parsing, markdown rendering via `marked.js` , code highlighting via `highlight.js` , Web Speech API voice dictation, and wizard steps.
`.env`	Environment Config	Stores private API keys (5 Gemini keys, Groq key, 6 OpenRouter keys, RapidAPI key) and HighLevel sub-account credentials.
`requirements.txt`	Python Dependencies	Pinned production package dependencies (FastAPI, Uvicorn, Google-GenAI, Requests, Python-Dotenv, Pydantic, Playwright).

3. Multi-Provider AI Model Infrastructure & Key Pools

To ensure 100% uptime for high-traffic agency environments, XortLogix implemented a triple-layer provider redundancy pool. The system operates 5 top verified models backed by 11 API keys:

Model Identifier	Provider	Pool Architecture	Rate Limits & Quota	Primary Use Case & Strengths
`gemini-3.6-flash` *(Default Engine)*	Google Gemini	5 Active Keys Pool (Round-Robin Rotation)	75 RPM 5,000,000 TPM 7,500 Requests/Day	Multimodal comprehension, direct GHL tool execution, sub-second latency (~300ms TTFT), full code generation.
`gemini-3.7-flash`	Google Gemini	5 Active Keys Pool (Round-Robin Rotation)	75 RPM 5,000,000 TPM 7,500 Requests/Day	Advanced reasoning engine configured for complex CRM workflow logic, data schemas, and mathematical calculations.
`groq/compound-mini`	Groq Cloud	Dedicated Groq LPU Key	30 RPM 70,000 TPM 14,400 Requests/Day	Ultra-high-speed inference on Groq Language Processing Units. Ideal for instant short-form CRM Q&A and tool calls.
`qwen/qwen3.8-27b`	Groq Cloud	Dedicated Groq LPU Key	30 RPM 70,000 TPM 14,400 Requests/Day	Open-weights code powerhouse. Acts as the primary seamless handover receiver when Gemini hits quota limits.
`meta-llama/llama-3.3-70b-instruct`	OpenRouter	6 Active Keys Pool (Gateway Failover)	~60-90 RPM Combined High Daily Token Cap	Ultra-resilient tertiary fallback model ensuring the application never produces a hard error even under severe provider outages.

3.1. Thread-Safe Key Pool Manager (`key_pool_manager.py`)

The `GeminiKeyPool` and `OpenRouterKeyPool` classes manage key rotation without race conditions:

- **Health Tracking:** Each key tracks its failure count, last success timestamp, and cooldown status.
- **Automated 429 Cooldown:** When an API key encounters an HTTP 429 (Rate Limit Exceeded) or `RESOURCE_EXHAUSTED` error, it is automatically marked depleted and isolated for 60 seconds. Subsequent requests instantly use the next healthy key in the pool.
- **Round-Robin Load Balancing:** Requests rotate across healthy keys sequentially to prevent any single key from exhausting its 15 RPM free-tier threshold.

3.2. Mid-Stream Model Handover & Seamless Continuation

When generating comprehensive marketing architectures containing thousands of tokens of HTML, CSS, JavaScript, and workflow tables, token limits can be reached mid-generation. The engine implements a stateful continuation mechanism:

1. **Truncation Detection:** The `detect_truncation()` function analyzes the stream to verify if HTML tags (`</html>`) remain unclosed, markdown code blocks remain unclosed, or workflow sections were cut off prematurely.
2. **Auto-Continuation (Same Model):** The model is re-invoked with the last 2,400 characters of generated context, instructed to continue from the exact last 80-character cutoff without repeating previous text.

3. **Cross-Model Handoff (Different Provider):** If a Gemini key pool is completely exhausted mid-stream, the engine prints a live handover notice in the stream:

   ```
   > 🔄 **Model Handover:** Google Gemini limit reached (3,420 chars generated). Seamlessly continuing with **Groq Cloud (Qwen 3.8)** from this exact point...
   ```

   The Groq model receives the preceding context and completes the remaining HTML tags and CRM tables without interruption.

4. GoHighLevel (GHL) Autonomous Tool Calling Engine

The copilot integrates with HighLevel API 2.0 (version `2021-07-28`) through native LLM function calling declarations. The agent autonomously decides when to invoke GHL tools based on user intent:

Function Name	GHL REST Endpoint	Parameters & Business Execution Logic
`create_contact`	`POST /contacts/`	Accepts `firstName`, `lastName`, `email`, `phone` (strict E.164 format), and `tags`. Automatically checks for duplicates in the sub-account.
`get_contact`	`GET /contacts/{id}`	Fetches complete contact profiles, custom field mappings, assigned campaigns, and active pipeline stages.
`create_pipeline`	`POST /opportunities/pipelines`	Accepts `name` and an ordered array of `stages`. Generates visual Kanban sales pipelines inside HighLevel Opportunities.
`create_opportunity`	`POST /opportunities/`	Accepts `pipelineId`, `stageId`, `contactId`, `title`, `monetaryValue`, and `status` (open, won, lost, abandoned).
`create_tag`	`POST /locations/{id}/tags`	Creates global classification tags (e.g. `lead:vsl-optin`, `abandoned:checkout`, `customer:core-member`).
`create_custom_field`	`POST /locations/{id}/customFields`	Creates custom fields with specific data types (`TEXT`, `NUMBER`, `SINGLE_OPTIONS`, `DATE`) for tracking video progress and tokens.

5. Conversion Funnel & CRM Engineering Standards

In response to rigorous agency production audits, **Muhammad Okasha** established strict architectural directives in `agent_engine.py` that govern how funnels, checkouts, and automations are generated:

5.1. Strict Entity Preservation Mandate

The AI engine is strictly constrained from swapping or genericizing user-provided business details. If a user requests a funnel for "*Mastermind Coaching Academy*" at $997 Core and $497 VIP Upgrade, the engine is forbidden from substituting generic examples (such as home services or roofing). All headlines, pricing, currencies, and copy must strictly reflect the user's prompt.

5.2. True 2-Step Order Form Validation

Rather than generating superficial single forms with mock text, the engine produces genuine 2-step order forms:

- **Sub-Step 1 (Contact Capture):** Collects First Name, Last Name, Email, and Phone. Clicking "*Continue to Payment*" performs client-side validation, applies the `intent:checkout-started` tag, and reveals Sub-Step 2.
- **Sub-Step 2 (Payment Authorization):** Contains credit card inputs (Card Number, Expiry, CVC, Zip) with real client-side validation (16-digit length check, expiration format check, CVC check). **Empty submissions are blocked with descriptive error messages.**
- **Dynamic Order Bump:** Includes an interactive checkbox (e.g. +$47 DM Playbook) that updates the itemized total in real time.

5.3. Real HTML5 Video Progress Tracking

Rather than using arbitrary timer intervals, the generated VSL rooms utilize actual HTML5 `<video>` elements with JavaScript `timeupdate` listeners. When the viewer crosses the **80% watch threshold**, the script dynamically unlocks the enrollment button and dispatches an HTTP POST event to a HighLevel Inbound Webhook to update `contact.vsl_completed = Yes`.

5.4. Separated Cart vs Lead Recovery Workflows

The architecture enforces separate, clean automations instead of blending lead reminders with cart abandonments:

- **Workflow 1 (Instant Access):** Opt-in form submitted ➔ Add `lead:vsl-optin` ➔ Send SMS with signed magic link + Confirmation Email.
- **Workflow 2 (24-Hour VSL Replay Cadence):** Wait 2h ➔ Wait 6h (Total 8h) ➔ Wait 16h (Total 24h) with stop-checks if checkout is initiated.
- **Workflow 3 (2-Step Order Form Cart Abandonment):** Triggered on Sub-Step 1 completion ➔ Wait 15m (SMS #1 if unpurchased) ➔ Wait 3h45m (Email #2) ➔ Wait 20h (Final Urgency Notice) with strict `customer:core-member` exit conditions.
- **Workflow 4 (Core & OTO Fulfillment):** Payment Received ($997) ➔ Add `customer:core-member`, remove `abandoned:checkout`. OTO Payment ($497) ➔ Add `customer:vip-upgrade`.
- **Workflow 5 (Dual-Event Onboarding Activation):** Moves opportunity to "*Onboarding Completed*" only when Calendar Appointment is Confirmed AND Member Portal Access is granted.

6. Frontend User Interface & Smart Asset Wizard

The frontend is constructed using pure vanilla HTML5, CSS3, and JavaScript without third-party frameworks:

- **Floating Input Island:** A centralized floating glassmorphism island featuring auto-growing textareas, file attachment previews (PNG, JPG, PDF, CSV, TXT, JSON), speech-to-text dictation, and a live pulsing model health indicator.
- **Interactive Suggestion Chips:** Quick-action pills located above the island for immediate execution: `🚀 Build Landing Page`, `+ Contact`, `+ Pipeline`, `+ Tag`, `+ Custom Field`, and `💡 Explain Funnels`.
- **6-Step Smart Asset Wizard Modal:**
  - *Step 1:* Niche & Business Model (Fitness, Real Estate, SaaS, Coaching, Dental, Solar, E-Commerce).
  - *Step 2:* Visual Design Aesthetic (Modern Dark Mode, Clean Corporate, High-Energy Neon, Luxury Slate & Emerald).
  - *Step 3:* Funnel Architecture (2-Step VSL, Lead Magnet Opt-in, Webinar, Application Funnel).
  - *Step 4:* Connected Automations & Workflows (SMS delivery, cart abandonment, review requests).
  - *Step 5:* Brand Customization (Business Name, Hero Tagline, Primary Color Picker with Hex presets).
  - *Step 6:* Review & Instant Single-File Bundle Generation.

7. Production Deployment & Operational Guide

The system is designed for instant containerized deployment on Railway, Render, AWS, or local developer environments:

```
# Step 1: Clone Repository
git clone https://github.com/muhammadokashapak/Conversation-AI-Copilot.git
cd Conversation-AI-Copilot

# Step 2: Install Dependencies
pip install -r requirements.txt

# Step 3: Configure Environment Variables (.env)
GEMINI_API_KEYS=key1,key2,key3,key4,key5
GROQ_API_KEY=your_groq_api_key
OPENROUTER_API_KEYS=key1,key2,key3,key4,key5,key6
GHL_LOCATION_ID=your_subaccount_location_id
GHL_ACCESS_TOKEN=your_private_integration_token

# Step 4: Run Application Server
python app.py
# Server binds to http://127.0.0.1:7861 with auto-reloading
```

> **Report Authentication:** Engineered and authored by **Muhammad Okasha** for **XortLogix**. Generated directly from verified repository source files. All architectural specifications, endpoints, key pools, and workflow automations are current, tested, and operational.